# Distributed Deep Learning With Gradient Compression for Big Remote Sensing Image Interpretation

Weiying Xie, *Senior Member, IEEE*, Jitao Ma, Tianen Lu, Yunsong Li, *Member, IEEE*, Jie Lei, *Member, IEEE*, Leyuan Fang, *Senior Member, IEEE*, and Qian Du, *Fellow, IEEE*

***Abstract*— Fast and reliable interpretation of high-dimensional hyperspectral images (HSIs) can provide great support to remote sensing-based Earth observations. Targets of interest in HSI can be detected using deep neural networks (DNNs) for background learning on an acquired image where the occurrence probability of background samples is much greater than that of targets, accounting for more than 95% of the whole scene. However, there is an increasing gap between theory and feasible application, because of the contradiction between massive hyperspectral data and resource-limited Internet of Things (IoT)/edge device hardware like satellite. To facilitate the deployment of hyperspectral target detection (HTD) in an edge computing environment, we introduce distributed background learning—a decentralized deep learning approach to meet the computing requirements of exploding high-dimensional data and larger DNNs. To address the communication bottleneck caused by gradient exchange during distributed learning, the proposed gradient compression solution, named gradient compression via centroid (GCC), uniquely compresses the most replaceable gradients with redundant information, thereby reducing communication overhead while maintaining accuracy. To illustrate the feasibility of the proposed method, we test it over two very large hyperspectral datasets with a total size of about 3.2 gigabytes (GBs) on a distributed system based on Ring All-reduce. We show that HTD based on distributed background learning outperforms those developed on a single node in terms of speed. Besides, the GCC compresses 50% gradients with only 0.01% loss of target detection accuracy to greatly reduce the communication overhead, surpassing existing gradient compression methods. It is expected that this framework will accelerate the introduction of distributed training on IoT/edge devices.**

***Index Terms*— Distributed deep learning, gradient compression, hyperspectral images (HSIs), large-scale/volume data, target detection.**

## I. Introduction

IN TERMS of scale and volume, remote sensing images have entered the era of big data, especially hyperspectral images (HSIs) with hundreds of spectral bands and complex spatial information [1], [2]. The well-known EO-1 satellite captures more than 1.6 TB of HSIs per day. How to efficiently gain insights into such a large volume of HSI is not only a key issue in improving the comprehensive application ability of spaceborne images but a challenge in computer science since the heavy computational and communication loads required.

Recently, significant advances have been made in HSI interpretation, especially in the hyperspectral target detection (HTD) task [3], [4], [5], thanks in part to larger and more sophisticated deep neural networks (DNNs) learned on massive HSIs. Particularly, a background learning-based hyperspectral target detector has been built, which takes a large number of background samples as training data and then uses the residual between the reconstructed HSI and the original one to highlight targets [6]. The diagram in Fig. 1 examples a typical background learning scheme. In general, the number of background samples accounts for more than 95% of a real HSI. For instance, in the typical HSI, 5 759 353 pixels belong to the background, accounting for 97.2% of the whole scene.

Notably, there is an increasing gap between what is technically feasible in theory and practical application, because of the contradiction between massive background data and resource-limited Internet of Things (IoT)/edge device hardware. Fig. 2 provides an IoT/edge use-case diagram and features of each part. Low latency requirement is increasingly being met, spanning from cloud servers, edge servers, to the end-to-end IoT/edge devices like satellites. Moreover, processing data on the satellites closest to the data source is beneficial to timely feedback and reduction of satellite-to-ground transmission cost, thus ensuring reliable and efficient use of bandwidth. However, it is difficult for these resource-constrained devices with very low performance individually struggle to deal with large volumes of HSIs that exceed gigabytes (GBs) in size on their own, especially for complex DNN-based HSI interpretation tasks.

Received 24 July 2022; revised 9 April 2023, 28 October 2023, 23 March 2024, and 13 September 2024; accepted 8 December 2024. Date of publication 23 December 2024; date of current version 6 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62322117, Grant 62371365, Grant U24B20136, and Grant U22B2014; in part by the Youth Talent Promotion Project of China Association for Science and Technology under Grant 2020QNRC001; and in part by the Fundamental Research Funds for the Central Universities under Grant YJSJ24017. *(Corresponding author: Jitao Ma.)*

Weiying Xie, Jitao Ma, Tianen Lu, Yunsong Li, and Jie Lei are with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: wyxie@xidian.edu.cn; jtm@stu.xidian.edu.cn; ysli@mail.xidian.edu.cn; jielei@mail.xidian.edu.cn).

Leyuan Fang is with the College of Electrical and Information Engineering, Hunan University, Changsha 410082, China (e-mail: fangleyuan@gmail.com).

Qian Du is with the Department of Electronic and Computer Engineering, Mississippi State University, Starkville, MS 39759 USA (e-mail: du@ece.msstate.edu).

Digital Object Identifier 10.1109/TNNLS.2024.3517535

2162-237X © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

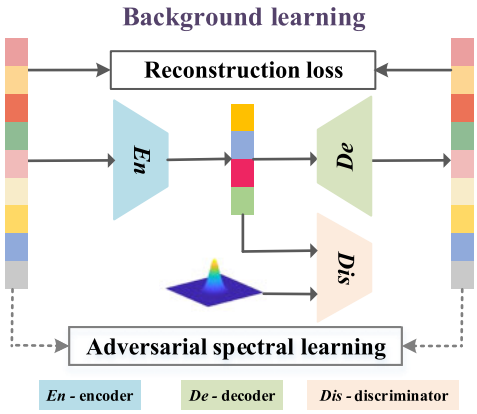



Fig. 1. Typical background learning scheme. After training on the dataset only with background spectral samples, the model can better represent background distribution, but not the low-probability targets.

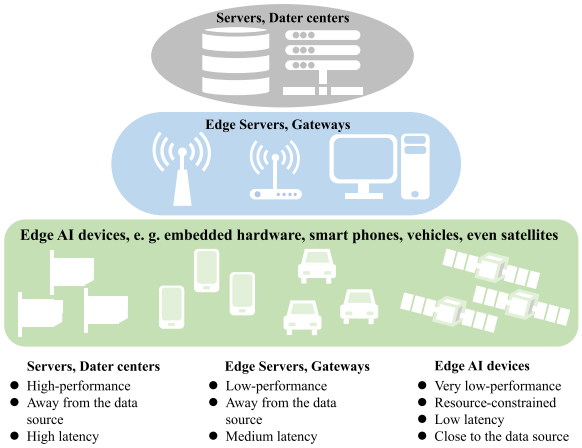



Fig. 2. IoT/edge use-case diagram and features of each part. A typical use case in an edge computing environment is satellite, involving edge training and inference, where reliability, accuracy, and latency are particularly important for decision-making on urgent missions.

This promotes distributed training by which DNNs can be replicated and background training data can be partitioned across distributed clusters [7], [8]. The commonly used distributed computing model, named MapReduce [9], has been initially explored and applied to HSI dimension reduction and HSI classification tasks to support deep learning on HSI big data [10], [11]. Very few work on HTD use distributed computing. This is partly due to the new challenge in communication bottleneck posed by distributed deep learning, where the parameters of DNNs (e.g., 525 and 232 MB for VGG16 [12] and AlexNet [13], respectively) are transmitted among edge nodes in a peer-to-peer fashion through capacity-limited communication channels, imposing a huge communication load on the network. Fig. 3 reports the comparison of communication time and computation time, and more nodes lead to a larger communication/computation ratio.

A series of efforts has been dedicated to overcoming the communication bottleneck in distributed training workloads via gradient compression. Recent developments on gradient compression can be divided into four categories, i.e., quantization [14], [15], sparsification [16], [17], [18], hybrid methods [19], [20] and low-rank methods [21], [22]. Among them, sparsification eliminates the number of elements

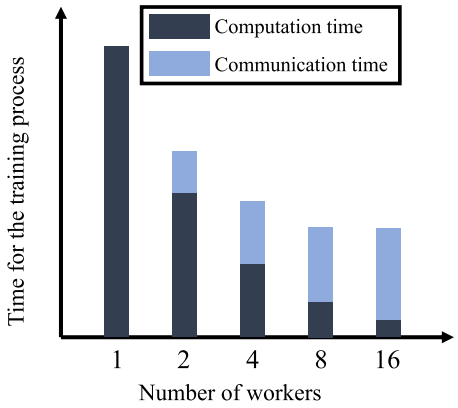



Fig. 3. Communication/computation ratio becomes larger as more nodes are used for training.

transmitted and shows great promise in various emerging distributed collaborative applications. Most of them exchange gradients that exceed a certain threshold. Overall, a portion of the gradient tensor is cleared so that zero-valued elements do not need to be transferred. To some extent, these methods tend to mitigate the problem of communication bottleneck. However, due to the reliance on "larger elements" criterion, some information may be lost because smaller elements do not mean unimportant.

To establish a novel compression criterion to overcome the communication bottleneck without a negative impact on the convergence, we systematically discuss the most replaceable elements from the perspective of information redundancy. Whether larger or smaller, the most replaceable elements mean absolutely unimportant. In this way, we investigate a new aspect that has not been studied in previous gradient compression works: effective redundancy removal via centroid calculation of the gradient tensor space within the same node. The cornerstone theory is that the centroid represents the common characteristic, so the closer it is to the centroid, the easier it is to be replaced, that is, it is regarded as redundancy. After removing redundancy, the proposed gradient compression algorithm, named gradient compression via centroid (GCC), achieves 94.09% detection accuracy with 50% gradients compressed when conducting target detection on Xiongan HSI with 1580 × 3750 pixels and 256 channels, exceeding existing gradient compression methods. More significantly, our gradient compression method GCC can be well-generalized to other distributed deep learning-based tasks. For instance, the gradient tensor of VGG16 is reduced by 20% with no Top-1 classification accuracy loss and ResNet20 still outperforms with pruning 50% gradients in the classification task.

In particular, our approach performs operating system (OS)-level visualization (i.e., Docker), which opens up more flexible ways for easy deployment. The designed docker container consists of the pretrained distributed background learning model, huge HSI dataset, code to GCC, and essential software environment including CUDA, OpenMPI, NCCL, PyTorch, and Horovod.

In summary, our main contributions include the following.

1) A distributed background learning model is designed for HSI target detection with large scenes for the first

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

time. Specifically, different from traditional distributed training for natural images, the proposed model focuses on exploring the global properties of HSI with large scenes so as to avoid boundary effects and frequent communication among capacity-limited communication channels through gradient accumulation.

2) The GCC is proposed to remove the most replaceable gradients with redundant information during distributed background learning, which can reduce communication costs while maintaining performance in remote sensing image interpretation tasks.
3) The GCC facilitates to easily and conveniently implement diverse DNN-based tasks on the nodes at the edge, which makes the proposed method naturally offer great feasibility for real applications with Ring All-reduce-based distributed system.
4) Extensive experiments validate that the GCC can significantly reduce redundant gradients with slight or even no loss of accuracy when being applied to big data classification task.

This article is organized as follows. Section II reviews background. Section III discusses our proposed distributed background learning for HTD. Section IV presents the proposed GCC. Section V details implementation and evaluation. Section VI includes discussion and concluding remarks.

## II. Background and Motivation

### A. Data-Parallel-Distributed Training

Most previous work on DNN training in parallel across multiple compute nodes can be divided into two categories, namely, model-parallel and data-parallel [23], [24]. In data-parallel distributed training, a set of nonoverlapping training samples is distributed among compute nodes, and each node calculates the gradients on its subset before aggregating the gradients in a centralized or decentralized manner. As illustrated in Fig. 4(a), the centralized approach consists of a server and several workers, where the server gathers gradients from each worker. In this case, the server must have significant computing and communication resources, and the downtime of the server will cause the entire training to be terminated. Although there is no affect on the performance of neural network model during training, the decentralized approaches in Fig. 4(b), such as Ring All-reduce [25] and 2D-HRA [26], transmit gradients among all nodes, bring about the problem of frequent point-to-point communication [27], [28], [29], [30], [31]. This is even more problematic for remote sensing satellites where communication is not as stable as on the ground.

### B. Distributed Computing in HSIs

HSI processing is a big data problem in terms of the resources required for acquisition, storage, management, and analysis. To address this problem, distributed computing-based methods are the major techniques for existing findings of studies. Two supervised machine learning methods, i.e., random forest and support vector machines, were presented for HSI classification [32], with the MapReduce [9] programming

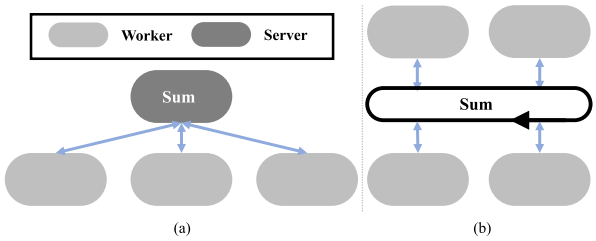



Fig. 4. (a) Leader–worker distributed learning. (b) Decentralized distributed learning.

model in Apache Hadoop framework [33]. Besides HSI classification, MapReduce is also adopted to implement the principle component analysis for HSI dimensionality reduction [34], [35] and Reed–Xiaoli (RX) detector for HSI anomaly detection [36]. More recently, distributed DNNs are developed to analyze HSIs [37], [38]. For instance, the unsupervised autoencoder (AE) enables to compress the hyperspectral data with a lower dimensional latent space across distributed clusters [37]. To take full advantage of large-scale GPUs for parallel training, a multi-GPU training framework was proposed to achieve cube-CNN-based HSI classification [7].

The above-distributed DNNs can alleviate the heavy computational load required to interpret big remotely sensed HSIs [11]; however, the lack of research to reduce storage space and communication bottleneck still has extra energy/power overhead risks, limiting the feasibility and flexibility of real HSI application. Therefore, it is urgent to explore novel distributed DNNs with efficient communication for learning from massive background samples.

### C. Gradient Compression

Efficient communication across distributed clusters has been widely concerned [39], where popular techniques include gradient quantization, sparsification, hybrid methods, and low-rank methods. Among them, gradient sparsification is more conducive to efficient communication while maintaining model performance and has broad application prospects. Strom made an earlier study on gradient sparsification, only transferring the gradient data larger than a certain threshold [40]. Sparse ternary compression (STC) [41], a new compression framework, is specifically designed to meet the requirements of federal learning environments. However, it is difficult to find an appropriate threshold in practice [42]. A multistage threshold-based compressor was proposed by imposing a sparsity prior on the gradients [43]. Considering that the variance of the gradient affects the convergence speed, a sparse coding method was presented to control the variance and make gradient sparsity [17], [44]. Wiedemann et al. [45] proposed a new data structure to compress and accelerate the weight matrix of DNNs to reduce the memory and algorithmic complexity of the inference process. Lin et al. [46] proposed deep gradient compression (DGC) to reduce communication bandwidth and applied it to image classification, speech recognition, and language modeling. Based on the observation that gradients values are mostly between $-1.0$ and 1.0, Li et al. [23] proposed a lightweight lossy-compression algorithm for image classification. On the

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

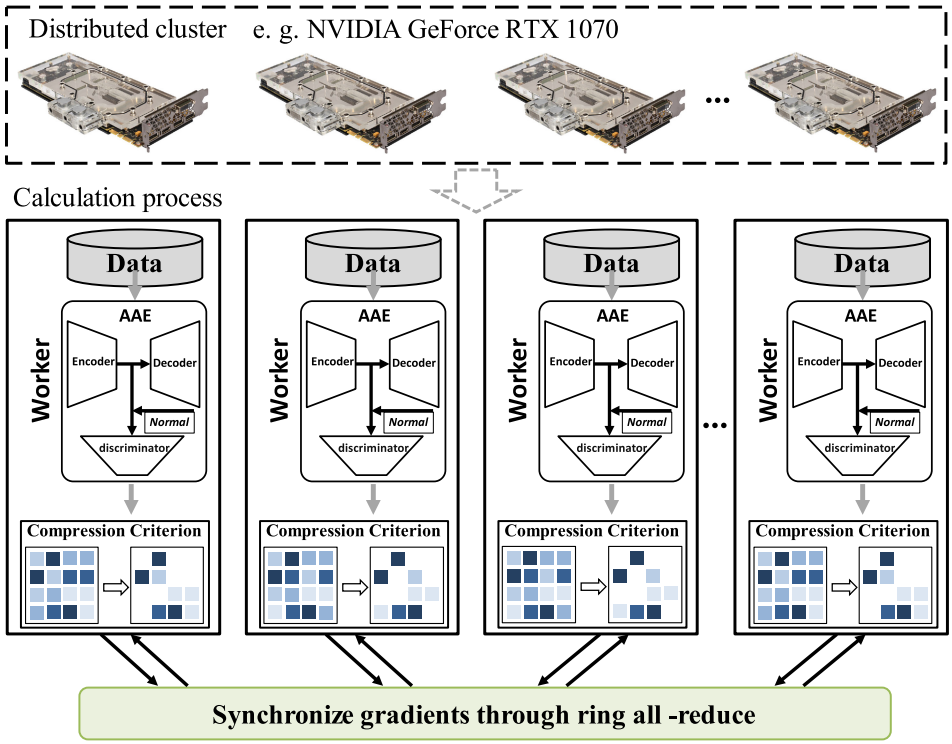



Fig. 5. Schematic of distributed background learning for HTD. The training set is divided into parts according to the number of workers and used to train the model (local AAE) on each node. The structure of local AAE is the same at each worker; after generating the gradient tensor, the gradients close to the centroid are zeroed. All workers synchronize gradients through the Ring All-reduce algorithm and then update parameters.

image classification task, the learned gradient compression (LGC) achieved the highest compression ratio [47]. Zhou et al. [48] proposed a bidirectional error feedback with lazily aggregated gradient (BiEF-LAG) algorithm to reduce the communication overhead of the uplink and downlink, addressing the dependence of complex model architectures that often leads to nonconvex distributed optimization. Zhou and Wang [49] designed a communication-efficient Lanczos algorithm (CELA). The clustering results are obtained using a distributed linear $k$-means algorithm based on low-dimensional approximate vectors. Jin et al. [50] tackled the communication overhead in distributed training of DNNs by refining the signSGD algorithm, which struggled with convergence under data heterogeneity in federated learning scenarios. They introduced a magnitude-driven stochastic-sign gradient compressor that resolved the convergence issue, while also quantifying Byzantine resilience.

This category of gradient compression methods has been widely employed in various distributed deep learning tasks. Our method is inspired by this line of research and also aims to build a specific DGC framework for learning from a large number of big backgrounds. Yet, what we more expect is to thoroughly and simply sparsity-redundant and replaceable gradients without loss of performance, which is different from the existing gradient sparsification methods.

## III. Distributed Background Learning for HTD

In general, an HSI covers hundreds of bands, which makes them hundreds of times larger than normal images with the same number of spatial pixels (see Section V-B). Therefore, distributed training is required to learn from a large number of normal background samples. Considering that the input data of HTD are large in size but the network model is relatively small, parallel computing is the powerful solution to this problem. In this parallel manner, each worker maintains its own model. In this section, we introduce our distributed AE model.

### A. Distributed AE Model

Inspired by the recent advances in self-supervised representation learning, AAE is chosen as the background learning backbone. As shown in Fig. 5, in a distributed cluster with $N$ workers, the input data are divided into portions based on the number of workers and the preset batch size BS. The segmented data will be sequentially input into the same AAE network on different workers. For worker $n$, the input is expressed as

$$\mathbf{X}_{\mathrm{BS}}^{n} = \left[x_1^n, x_2^n, x_3^n, \ldots, x_{\mathrm{BS}-1}^n, x_{\mathrm{BS}}^n\right] \in \mathbb{R}^{\mathrm{BS}\times B} \tag{1}$$

where $B$ denotes the number of bands.

The background learning model includes encoder $\mathbf{En}(\cdot)$, decoder $\mathbf{De}(\cdot)$, and discriminator $\mathbf{Dis}(\cdot)$, as shown in Fig. 1, all of which are composed of multilayer fully connected neural networks with a tanh activation function. For worker $n$, $\mathbf{En}(\cdot)$ aims to map the input spectrum $x_i^n$ as a low-dimensional representation $c_i^n$ with the parameter $\theta_{\mathrm{en}}$

$$c_i^n = \mathbf{En}\left(x_i^n, \theta_{\mathrm{en}}\right) \tag{2}$$

and $\mathbf{De}(\cdot)$ is used to reconstruct $x_i^n$ with $c_i^n$

$$y_i^n = \mathbf{De}\left(c_i^n, \theta_{\mathrm{de}}\right) \tag{3}$$

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

where $\theta_{\text{de}}$ is the parameter of $\mathbf{De}(\cdot)$, and $y_i^n$ denotes reconstruction result. The loss function of the reconstruction error can be expressed as

$$\mathcal{L}_{\text{AE}}^n = \underbrace{\frac{1}{\text{BS}}\sum_{i=1}^{\text{BS}}\left\|x_i^n - y_i^n\right\|^2}_{\mathcal{L}_{\text{RS-AE}}^n} + \underbrace{\sum_{i=1}^{\text{BS}}\cos^{-1}\left(\frac{y_i^{nT} d}{\sqrt{d^T d}\sqrt{y_i^{nT} y_i^n}}\right)}_{\mathcal{L}_{\text{TS}}^n} \tag{4}$$

where $d$ is a known sample of the target. The first component, $\mathcal{L}_{\text{RS-AE}}^n$, refers to the reconstruction error caused by the AE. The second component, $\mathcal{L}_{\text{TS}}^n$, refers to the loss of the target suppression constraint imposed on the ambiguity of the background samples obtained by the traditional method. Here, $\mathbf{Dis}(\cdot)$ is used to determine whether the input is true or false to instruct $\mathbf{En}(\cdot)$ to generate a better low-dimensional representation according to

$$\text{real}_i^n = \mathbf{Dis}(\mathbf{I}, \theta_{\text{dis}}) \tag{5}$$

$$\text{fake}_i^n = \mathbf{Dis}\left(c_i^n, \theta_{\text{dis}}\right) \tag{6}$$

where $\theta_{\text{dis}}$ is the parameter of $\mathbf{Dis}(\cdot)$, and $\mathbf{I}$ is a priori Gaussian distribution. The loss function of the discriminator can be expressed in the form of

$$\mathcal{L}_{\text{Dis}}^n = \frac{1}{\text{BS}}\sum_{i=1}^{\text{BS}}\left(\text{cross}\left(\text{real}_i^n, \text{ones}\right) + \text{cross}\left(\text{fake}_i^n, \text{zeros}\right)\right) \tag{7}$$

where cross$(\cdot)$ stands for cross-entropy, ones is a matrix in which all elements are 1, and zeros is a matrix where all elements are 0. The worker $n$ then performs backward propagation based on the calculated loss. Subsequently, the model parameters are optimized for the properties of HSIs and distributed training.

### B. HSI Data Distribution

In HSI interpretation tasks, a pixel in the spatial domain is typically used as input to the model, and each pixel is actually a vector in the spectral dimension. An HSI with $H \times W$ pixels and $B$ bands can be reshaped to

$$\mathbf{H}_o = [h_1, h_2, h_3, \ldots, h_{M-1}, h_M] \in \mathbb{R}^{M \times B} \tag{8}$$

where $M = H \times W$. However, $\mathbf{H}_o$ cannot be directly used for background learning and needs to be preprocessed. In this work, to reconstruct the background as accurately as possible, the traditional constrained energy minimization is leveraged as the coarse detection approach to obtain the initial training set. Specifically, $\mathbf{H}_o$ is divided into background samples set and target samples set, i.e.,

$$\mathbf{B} = [b_1, b_2, b_3, \ldots, b_{L-1}, b_L] \in \mathbb{R}^{L \times B} \tag{9}$$

$$\mathbf{T} = [t_1, t_2, t_3, \ldots, t_{Q-1}, t_Q] \in \mathbb{R}^{Q \times B} \tag{10}$$

where $\mathbf{B}$ and $\mathbf{T}$ denote the set of background spectral samples and set of target spectral samples, respectively. The number of background spectrum $b_i$ and target spectrum $t_i$ are $L$ and $Q$, respectively. Subsequently, we randomly select 75% of $\mathbf{B}$ as the training set $\mathbf{X}$, which serves as the input of our distributed background learning network.

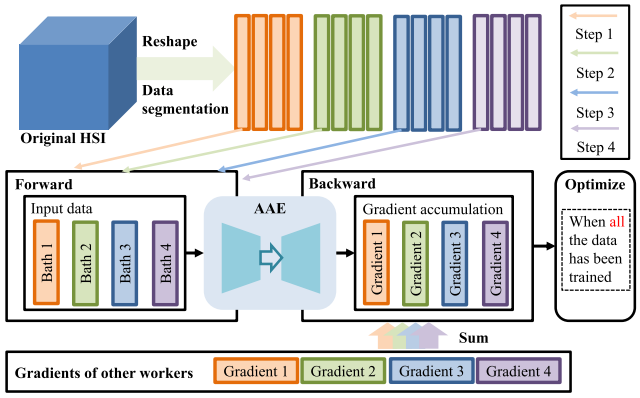



Fig. 6. Detailed training process in worker $n$.

### C. Gradient Accumulation

To reduce the communication frequency in distributed background training while avoiding boundary effects as much as possible, gradient accumulation is leveraged during training. Fig. 6 provides the computation process for worker $n$ in one epoch. At the start of training, worker $n$ feeds local data into the background learning network and performs forward and backward propagations to obtain the gradient tensor, $\text{grad}^n$. Meanwhile, other workers perform the same operations as worker $n$ until all input data have been trained and worker $n$ adds the new gradients to the previous gradients.

Subsequently, the workers in the distributed cluster transfer the gradients via peer-to-peer communication. Finally, all workers generate the same gradient and optimize the local background learning network by means. More specifically, the update rule for the trainable parameter $\boldsymbol{\theta}$ of a distributed AAE model with gradient accumulation can be written as

$$\mathcal{I}_t^n = \text{Accumulate}\left(\boldsymbol{\theta}_t, \boldsymbol{X}_{\text{BS}}^n\right) = \sum_{s=1}^{\text{BS}} \nabla\mathcal{L}_{\text{AE},t}^n\left(\boldsymbol{\theta}_t, x_{t,s}^n\right) \tag{11}$$

$$\mathcal{O}_t^n = \text{Gradient} - \text{Compress}\left(\mathcal{I}_t^n\right) \tag{12}$$

$$\boldsymbol{\mathcal{O}}_t = \left\{\mathcal{O}_t^1, \mathcal{O}_t^2, \ldots, \mathcal{O}_t^N\right\} \tag{13}$$

$$\mathcal{G}_t = \text{Gather}(\boldsymbol{\mathcal{O}}_t) = \frac{1}{N}\sum_{n=1}^{N}\mathcal{O}_t^n \tag{14}$$

$$\boldsymbol{\theta}_{t+1} = \text{Adam}(\boldsymbol{\theta}_t, \mathcal{G}_t, \alpha) \tag{15}$$

where $\nabla\mathcal{L}_{\text{AE}}^n(\boldsymbol{\theta}_t, x_{t,s}^n)$ denotes the gradient matrix with bath $s$ on node $n$, $\mathcal{O}_t^n$ denotes the accumulated and compressed gradient matrix on node $n$, $\mathcal{G}$ denotes the processed (accumulate, compress, and gather) gradient matrix, and $\alpha$ denotes the learning rate of the training process.

In this way, the results obtained are equivalent to the results of feeding the entire training set into the network, which can effectively improve the performance of background learning while employing multiple resource-constrained edge nodes to handle large background learning tasks.

### D. Gradient Synchronization

The inevitable exchange of gradients among numerous nodes during distributed background learning, also known as

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

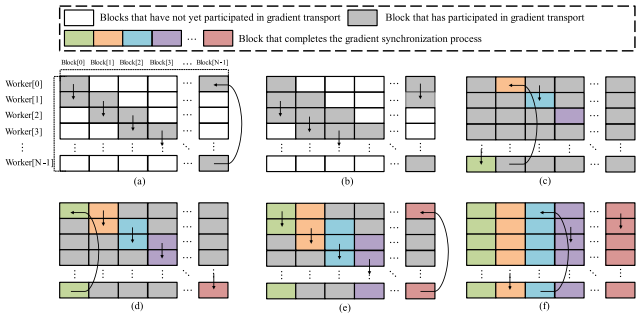



Fig. 7. Example of $N$ workers Ring All-reduce: (a) step 1: Scatter-reduce(1); (b) step 2: Scatter-reduce(2); (c) step $N-1$: Scatter-reduce($N-1$); (d) step $N$: All-gather(1); (e) step $N+1$: All-gather(2); and (f) step $2N-2$: All-gather($N-1$).

gradient synchronization, is the essential bottleneck that limits the training speed of data-parallel neural network models. Gradient synchronization is generally implemented by the All-reduce algorithm, and the Ring All-reduce algorithm is widely used because it can effectively relieve the communication bottleneck. In addition, the Ring All-reduce algorithm is a decentralized All-reduce algorithm. As shown in Fig. 7, in this algorithm, there is no leader worker in the distributed cluster. Instead, each worker exchanges gradients with two adjacent nodes after each iteration. Initially, each worker evenly divides the local gradient into $N$ parts. Taking the $n$th worker as an example, the gradient is divided according to

$$\text{grad}^n = \{\text{Block}[0], \text{Block}[1], \text{Block}[2], \ldots, \text{Block}[N-1]\}. \tag{16}$$

The Ring All-reduce algorithm consists of Scatter-reduce and All-gather, both of which need to be iterated $N-1$ times. The main purpose of Scatter-reduce is to exchange partial gradients and fuse so that each node will eventually contain a portion of the complete gradient. The main purpose of All-gather is to exchange incomplete fusion gradients so that a complete gradient is obtained at each node, as follows.

1) *Scatter-Reduce:* In the first step, worker $n$ sends Block[$n$] to its next node, worker $n+1$. If $n = N-1$, worker $n$ sends Block[$n$] to worker 0. Meanwhile, worker $n$ receives Block[$n-1$] from worker $n-1$ or receives Block[$N-1$] from worker $N-1$ if $n=0$. The received Block[$n-1$] will be added to the local Block[$n-1$]. In the next step, worker $n$ sends Block [$n-1$] to its next node, and so on, until worker $n$ has fully aggregated Block[$n+1$] from all other workers.
2) *All-Gather:* In the first step, worker $n$ sends Block[$n+1$] to its next node, worker $n+1$. If $n = N-1$, worker $n$ sends Block[$n+1$] to worker 0. Meanwhile, worker $n$ receives Block[$n$] from worker $n-1$ or receives Block[0] from worker $N-1$ if $n=0$. The received Block[$n$] will replace the local Block[$n$]. In the next step, worker $n$ sends Block[$n$] to its next node, until worker $n$ has fully aggregated all the Block[$i$] from worker $i-1$, respectively, $i = 0, 1, \ldots, n, n+2, \ldots, N-1$. Algorithm 1 formally describes the Ring All-reduce procedure.

In addition, the detection performance of our model is robust to different connectivity methods, as the model ultimately relies on the integrity of the transmitted information rather than the specifics of the transmission path [33], [51], [52], [53]. In summary, Ring All-reduce effectively utilizes the bandwidth of each worker and avoids the communication bottleneck caused by the leader node in a leader–worker architecture. To further reduce frequent peer-to-peer communication, we introduce a novel gradient compression criterion in Section IV.

**Algorithm 1** Calculation of Worker[$n$] in an $N$-Worker Cluster

**Initialize:** the same model with weights $w_0$
**for** *epoch* = 0,...,(*epochs*-1) **do**
  **If not** all the data is trained **then**
    $\mathbf{X}^n_{BS}$ = Load a mini-batch
    $\mathcal{L}^n_{AE}, \mathcal{L}^n_{Dis} = Forward(\mathbf{X}^n_{BS})$
    Compute local gradient $grad^n = Backward()$
  // (Compress local gradient $grad^n$)
  $grad^n \leftarrow GCC(grad^n)$
  // Ring All-reduce
  **For** *step* = 0,...,(*N*-1) **do**
    Send Block$_{local}[n]$ to worker $(n+1)\ mod\ N$
    Receive Block$[(n-1)\ mod\ N]$ from worker $(n-1)\ mod\ N$
    Block$_{local}[(n-1)\ mod\ N]$ += Block$[(n-1)\ mod\ N]$
  **End for**
  **For** *step* = 0,...,(*N*-1) **do**
    Send Block$_{local}[(n+1)\ mod\ N]$ to worker $(n+1)\ mod\ N$
    Receive Block$[n]$ from worker $(n-1)\ mod\ N$
    Block$_{local}[n] \leftarrow$ Block$[n]$
  **End for**
  Optimize $w_0$ by $grad^n$
**End for**

## IV. Gradient Compression via Centroid

Although the existing All-reduce algorithm optimizes the gradient synchronization process in distributed training, the size of the transferred gradient data can still cause a communication bottleneck. In order to reduce the communication overhead of distributed background learning, we perform an experimental analysis of the gradient properties from a novel perspective in Section IV-A and propose the GCC algorithm to compress the gradients to be transmitted before Ring All-reduce in Section IV-B.

### A. Redundant Gradient Discussion

In order to reduce communication overhead and maximize the utilization of workers in distributed clusters, we incorporate gradient compression into distributed background learning. Most of the existing gradient compression methods take the value or norm of gradients as the compression criterion. These methods require two implicit conditions to be satisfied to achieve better results: 1) the value or norm of the relatively small gradient makes less contribution to model optimization and 2) the standard deviation of the value or norm

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

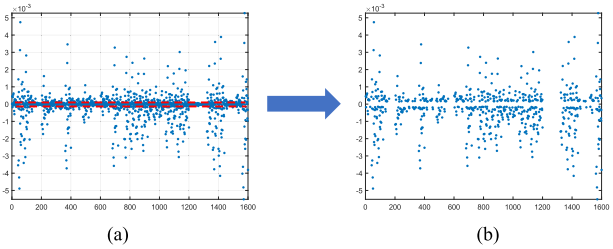



Fig. 8. Gradient compression (compression rate = 0.5) based on the value or norm of gradients: (a) gradients distribution before compression and (b) gradients distribution after compression. Note that the red dotted line represents the threshold for gradient compression.

of the gradient is sufficiently large. However, in the actual training process, the above conditions cannot be satisfied in all cases. Therefore, we first analyze the redundant relationship between the gradients to eliminate redundant gradients. Inspired by Fletcher et al. [54], we map the gradients to Euclidean space and compute the centroid of the gradient tensor space, which can be equivalent to the center of these gradients, or common property of these gradients. Gradients close to the centroid can be considered redundant compared to other gradients. Removing these redundant gradients does not have much impact on the optimization of the model parameters. The remaining gradients are considered as a set of bases that can be reconstituted into the removed gradients in different combinations and guide the model in the desired direction for optimization.

*1) Conventional Compression Criterion:* Gradient compression methods based on the value or norm of gradients usually require the selection of an appropriate threshold. As shown in Fig. 8, gradients below a set threshold will be set to zero or will not be transmitted. Typically, a larger gradient has a greater impact on the optimization of the model parameters, but this does not mean that a relatively small gradient is unimportant. A relatively small gradient is not necessarily because its value or norm is too small, but it could also be that all the gradients in an iteration are large, as illustrated in Fig. 8. In this case, removing the relatively small gradients would undoubtedly have a negative impact on the model.

In addition, when the standard deviation of the value or norm of gradients is small, it is difficult to choose a suitable threshold for gradient compression. Once the threshold is not chosen properly, important gradient information may be compressed away, which negatively affects model performance.

*2) Redundant Relationship as Compression Criterion:* To avoid the challenges discussed above, from an information redundancy perspective, the most replaceable gradients must be supported. Based on the assumption that the centroid in Euclidean space reveals the common attribute, we map gradient values to points in Euclidean space to analyze the redundant relationships between gradients.

In Fig. 9, to analyze the centroid properties of gradient, the transformations of (a) to (c) and (b) to (d) are formulated as

$$\gamma = \frac{2\pi}{t} \tag{17}$$

$$d_k^n[i, j] = \phi(g_t, d_k^n) \tag{18}$$

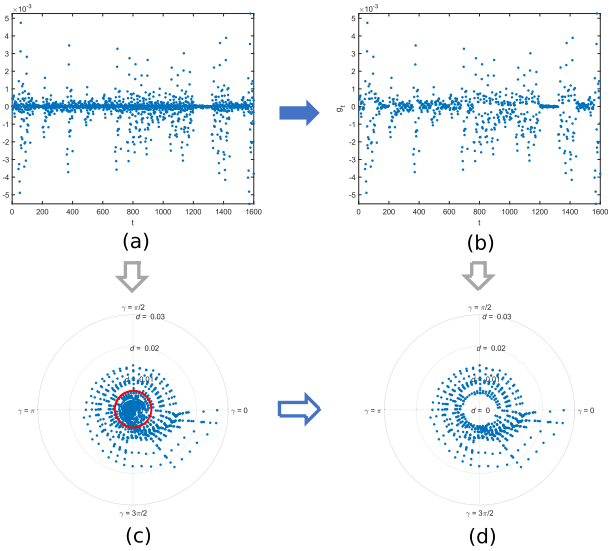



Fig. 9. Gradient compression (compression rate = 0.5) based on centroid: (a) gradients distribution before compression; (b) gradients distribution after compression; (c) distance from the gradients to the centroid before compression; and (d) distance from the gradients to the centroid after compression. Note that the red dotted circle indicates the threshold for gradient compression, with the centroid at the center of the circle.

where $t$ is the index representation of gradient, $d$ denotes the distance from the gradients to the centroid, $g_t$ corresponds to $\text{grad}_k^n[i, j]$, and function $\phi$ is defined as (21).

As illustrated in Fig. 9(a) and (c), most gradients have small and close distances to the centroid, and then, these gradients can be replaced by those farther away from the centroid. The red dotted circle in Fig. 9(c) corresponds to the red dotted line in Fig. 8, representing the compression rate of 0.5. The gradients inside the circle are closer to the centroid of the gradient tensor space, and they contain a high degree of similarity in information that can be reorganized from the gradients outside the circle. Consequently, their contributions are most easily replaced, and their removal does not substantially affect model performance. As demonstrated in Fig. 9(b) and (d), in contrast to those gradient compression methods based on the value or norm of gradients [see Fig. 8(b)], the result obtained by the centroid-based gradient compression method preserves both relatively large and relatively small gradients, thus retaining most of the important gradient information even when the two implied conditions mentioned above are not satisfied.

### B. Gradient Compression via Centroid

According to our experimental analysis in Section IV-A, we conclude that only the gradients from the closest centroid should be pruned. In this section, the proposed gradient compression algorithm, GCC, is specified.

As the centroid is a classic robust estimator of centrality for data in Euclidean space [54], we calculate the centroid as the common attribute of gradients of a full connection layer. Given the width of gradient matrix and the height of gradient matrix (i.e., $I$ and $J$), assuming that the gradients of the $k$th fully connected layer in worker $n$ is $\text{grad}_k^n \in \mathbb{R}^{I \times J}$, and

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

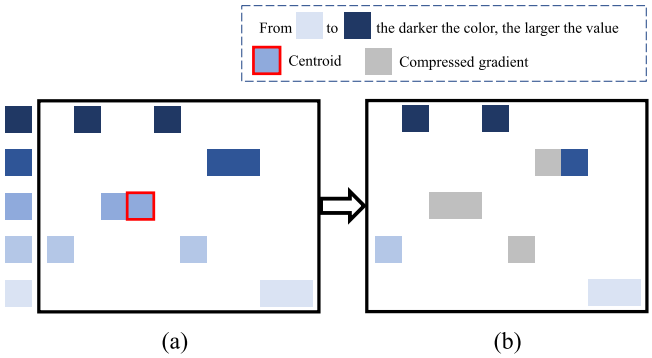



Fig. 10. Schematic of the proposed GCC algorithm: (a) gradients before compression and (b) gradients after compression.

then, the calculation of the centroid, $C_k^n$, of gradient matrix is

$$C_k^n = \underset{\text{grad}_k^n[i,j]\in\text{grad}_k^n}{\arg\min} \sum_{i,j=1}^{i=I,j=J} \left\| C_k^n - \text{grad}_k^n[i,j] \right\|_2 \tag{19}$$

where $\text{grad}_k^n[i, j]$ denotes the gradient in row $i$ and column $j$. The distance between all different gradients $d_k^n \in \mathbb{R}^{I*J}$ is

$$d_k^n = \text{sort}(d_k^n[1,1], \ldots, d_k^n[I,J]) \tag{20}$$

$$d_k^n[i,j] = \left\| C_k^n - \text{grad}_k^n[i,j] \right\|_2 \tag{21}$$

where sort is a sorting function in ascending order. Now, given $d_k^n$ and compression rate cr, the index $S$ of the smallest element is established as

$$S = \text{Find}\left(d_k^n, \text{cr}\right) \tag{22}$$

$$\hat{\text{cr}} = \frac{\sum_{m=1}^{S} d_{k,m}^n}{\sum_{m=1}^{I*J} d_{k,m}^n}, \quad \text{s.t.} \ \min_{\hat{\text{cr}}\geq\text{cr}} ||\hat{\text{cr}} - \text{cr}||_2^2 \tag{23}$$

where $\hat{\text{cr}}$ represents the actual compression rate. For a given compression rate cr, the gradients corresponding to the first int($I \times J \times$ cr) elements of $d_k^n$ will be set to zero, as shown in Fig. 10. Especially, in order to better map the gradients to Euclidean space while reducing the computational effort of gradient compression, we chose the gradient closest to the centroid instead of the centroid, effectively avoiding the computational cost required to calculate the centroid. Algorithm 2 describes the simplified GCC algorithm in detail.

The output of Algorithm 2 is partially zeroed gradients, which can still be used to guide the network to optimize in the desired direction since important information is retained. It should be noted that the geometric center also exists in the convolutional neural network, but to reduce the amount of computation, the high-dimensional gradient matrix needs to be expanded to two dimensions. Otherwise, the steps are the same as those of the fully connected layer. With GCC, we can easily eliminate redundant gradients and reduce the communication overhead without a significant impact on the optimization of the model.

**Algorithm 2** Algorithm Description of GCC

**Input**: $grad_k^n, cr$
// Working in $k_{th}$ layer of worker $n$
// Assuming that $I \geq J$
**For** $i = 1, 2, \ldots, I$ **do**
  **For** $j = 1, 2, \ldots, I$ **do**
    $norm[i, j] = \| grad_k^n[i, :] - grad_k^n[j, :] \|_2$
  **End for**
**End for**
// Compute the redundancy of the vectors
**For** $i = 1, 2, \ldots, I$ **do**
  $similar[i] = \sum_{j=1}^{I} norm[i, j]$
**End for**
$S = Find(similar, cr)$ as Equation 22 $\sim$ Equation 23
$grad_k^n[S, :] \leftarrow 0$
**Output**: $grad_k^n$

## V. Implementation and Evaluation

To validate the effectiveness and generality of the proposed method, we carry out target detection experiments on two widely used hyperspectral remote sensing datasets, including Xiongan New Area (Matiwan Village) and AeroRIT, and image classification experiments on a natural image dataset, i.e., CIFAR-10 dataset.

### A. Experimental Setup

*1) Experimental Environment:* All experiments are deployed on a distributed system that consists of two edge nodes running in the Linux operating system (Linux OS), Ubuntu 20.04. In particular, the hardware environment of each edge node contains an Advanced Micro Devices (AMD) processor operating at 2.2 GHz, 8 GB of RAM, a disk of 256 GB, and one GPU, NVIDIA GeForce RTX 1070. Data transfers are supported by a local 3 × 100 Mb/s dedicated network.

Meanwhile, in order to unify and deploy the training environment of each edge node, container technology (i.e., Docker 20.10.14) is developed as the running platform. OpenMPI 3.0.0, CUDA 11.2.2, Python 3.8, and Horovod 0.21 [33] are preinstalled in our designed Docker images.

The open-source distributed framework, Horovod for PyTorch 1.8 [55], employs efficient inter-GPU communication and provides a distributed machine learning library to support our distributed background learning network for HTD task. PyTorch includes multiple extensions (e.g., NVIDIA Apex) that enable streamlined mixed precision with distributed training. The engine and network constructors of the proposed gradient compression method (GCC) are built in pure Python and do not rely on other complex third-party libraries. The code is available at https://github.com/majitao-xd/GCC.

*2) Evaluation Metrics:* The receiver operating characteristic (ROC) curve is a 2-D depiction of the detector performance, which is about the tradeoff between the detection rate and the false alarm rate of the detector [56]. To represent the expected detector performance via a single scalar value rather than the ROC curve, a common method is to calculate the area under the ROC curve, referred to as area under curve (AUC) [57]. Given a segmentation threshold $\tau$, we specify that pixels whose anomaly scores greater than or equal to $\tau$

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

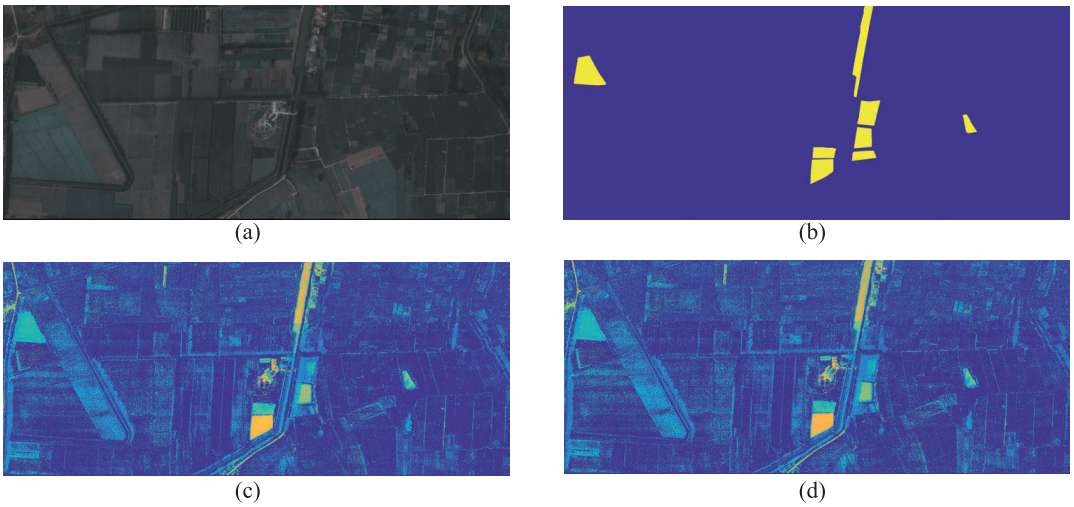


Fig. 11. Visual quality comparison on typical scenes in Xiongan New Area (Matiwan Village) dataset. (a) Pseudo-color image. (b) Ground truth of target class (i.e., water). (c) Baseline HTD result. (d) Detection result with gradient compression rates set to 0.5.

are classified as positive samples, and detection probability $F_0(\tau)$ and false alarm probability $F_1(\tau)$ can be calculated as

$$F_0(\tau) = \frac{\mathrm{TP}(\tau)}{\mathrm{TP}(\tau) + \mathrm{FN}(\tau)} \tag{24}$$

$$F_1(\tau) = \frac{\mathrm{FP}(\tau)}{\mathrm{TN}(\tau) + \mathrm{FP}(\tau)} \tag{25}$$

where $\mathrm{TP}(\tau)$ is the number of target pixels that are predicted as positive samples under threshold $\tau$, $\mathrm{FN}(\tau)$ is the number of target pixels that are predicted as negative samples, $\mathrm{FP}(\tau)$ is the number of background pixels that are predicted as positive samples, and $\mathrm{TN}(\tau)$ is the number of background pixels that are predicted as negative samples. Then, the false alarm rate (also known as the false positive rate, FPR) and the AUC are calculated according to

$$\mathrm{FPR} = \int_0^1 F_1(s)ds \tag{26}$$

$$\mathrm{AUC} = \int_0^1 F_0(s)dF_1(s). \tag{27}$$

For the target detection task on the HSI datasets, AUC and FPR are calculated to quantify background learning network performance. The larger the AUC value of $[F_0(\tau), F_1(\tau)]$, the higher the detection accuracy, and the smaller the AUC value of $[F_1(\tau), F_0(\tau)]$, the lower the false alarm rate.

For image classification task, the top-$k$ classification accuracy, notated as $\mathrm{ACC}_k$, is widely used, where $k$ is usually set to $\{1, 5\}$. Specifically, it is formulated by

$$\mathrm{ACC}_k = \frac{N_{C^k}}{N_{C^\tau}} \tag{28}$$

where $N_{C^k}$ denotes the number of correct labels in all test images included in the first $k$ classes, and $N_{C^\tau}$ denotes the number of all test images. Only $\mathrm{ACC}_1$ is used in our image classification experiments on the CIFAR-10 dataset.

### B. Data Description

*1) Xiongan New Area (Matiwan Village) Dataset [58]:* This aerial hyperspectral remote sensing image was acquired in October 2017 by the visible and near-infrared imaging spectrometer designed by Shanghai Institute of Technical Physics, Chinese Academy of Sciences. The dataset has 256 bands with wavelengths ranging from 400 to 1000 nm and 3750 × 1580 pixels with a spatial resolution of 0.5 m. This hyperspectral dataset comprises 19 land cover classes, of which water is considered as the target of interest and other classes served as background occupying nearly 0.896 and 31.104 GB of storage space, respectively. Table I lists details in terms of the number of pixels for the target of interest and noninteresting background, while Fig. 11(a) and (b) shows the false-color image and ground truth, respectively.

TABLE I

NUMBER OF PIXELS AND OCCUPIED RATIO OF INTERESTED TARGETS (I.E., WATER) AND BACKGROUND ON TWO HSI DATASETS INCLUDING XIONGAN NEW AREA (MATIWAN VILLAGE) AND AERORIT

| Dataset | Classes | No. of Pixel | Ratio (%) |
|---|---|---|---|
| Xiongan New Area (Matiwan Village) | Water | 165647 | 2.80 |
| | Background | 5759353 | 97.20 |
| AeroRIT | Water | 68576 | 1.50 |
| | Background | 4491424 | 98.50 |

*2) AeroRIT Dataset [59]:* The large-scale hyperspectral sensing image was acquired by two types of camera systems onboard a Cessna aircraft over the university campus of Rochester Institute of Technology. A framing-type silicon sensor working in the RGB domain with 80 megapixel (MP) is used in the first while a visible near-infrared (VNIR) hyperspectral Headwall Photonics Micro Hyperspec E-Series CMOS sensor is used in the second. The image contains 372 bands with wavelengths ranging from 397 to 1003 nm. AeroRIT consists of 1973 × 3975 pixels with nearly seven-million-pixel annotations but only includes five classes. In the evaluation experiments, the occupied memory for the target and background is close to 48.67 and 3196.13 Mb, respectively. In our experiments, the spatial positions of valid images are as follows: the top-left and bottom-right spatial positions of the original image are (401, 101) and (1600, 3900), respectively, while the valid spatial size is 1200 × 3800 for all bands. Table I and Fig. 12(a) and (b) provide the details.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

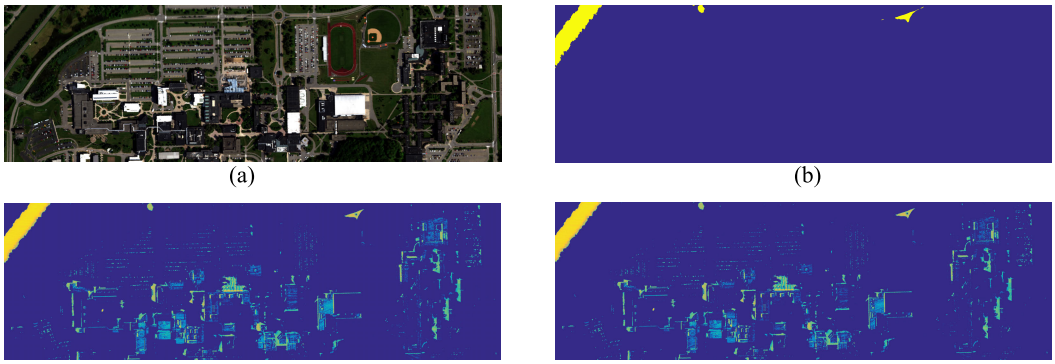



Fig. 12. Visual quality comparison on typical scenes in AeroRIT dataset. (a) Pseudo-color image. (b) Ground truth of target class (i.e., water). (c) Baseline HTD result. (d) Detection result with the gradient compression rate set to 0.5.

*3) CIFAR-10 Dataset [60]:* The small-sized dataset (nearly 163 MB) was collected by Krizhevsky and Hinton [60]. The natural image dataset consists of $6 \times 10^4$ color images including ten classes as follows: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. There are $5 \times 10^4$ training images, consisting of five training batches, and $1 \times 10^4$ test images (one test batch). In addition to the test images, the training batches contain the remaining images in a random order, and a training batch contains a variable number of 5000 images in each class.

### C. Results and Analysis

In this section, the proposed distributed background learning with gradient compression for the HTD task is examined. In addition, the proposed novel compression criterion, i.e., the GCC is generalized to the distributed image classification task. The performance is analyzed and validated by the subjective and objective results.

*1) Target Detection on HSI Datasets:*

*a) Target detection on Xiongan new area (Matiwan village) dataset:* Implementation details in terms of network architecture and parameter settings are shown in Table II, and the network is fully trained with the same settings in Section V-A. In the same experimental environment, the results under different compression rates (three times on average) are shown in Table III. We can see that the proposed method only achieves a 0.01% decrease in AUC score at a high gradient compression rate of 50%. Furthermore, when the gradient compression ratio is set to 0.2, the AUC score drops by even 0.00%, which indicates that the gradient sparsity is as high as 20% (only 80% is effective). Compared with DGC [46], where only a 92.63% AUC is obtained, the proposed GCC gains a better result of 94.09%. Lastly, compared with Top-$K$ [61], when the compression rate is set to 0.5, GCC still achieves higher AUC (93.38% versus 94.09%) and outperforms FPR (7.36% versus 7.67%).

Fig. 11 shows a comparison of visual results, including the pseudo-color image, ground truth of the target and background, and detection maps. The visual evaluation results are consistent with the objective results in Table III, which verifies the reliability and effectiveness of our method.

TABLE II
PARAMETER SETTINGS AND NETWORK ARCHITECTURE OF EXPERIMENTS ON DIFFERENT DATASETS

| Datasets | Network & Parameter | Value |
|---|---|---|
| Xiongan New Area (Matiwan Village) | Network | AAE |
| | Batch Size | 200000 |
| | Epoch | 200 |
| | Learning Rate (LR) | $1 \times 10^{-4}$ |
| | Momentum | - |
| | Weight decay factor | - |
| | Optimizer | Adam |
| AeroRIT | Network | AAE |
| | Batch Size | 310000 |
| | Epoch | 100 |
| | Learning Rate (LR) | $1 \times 10^{-4}$ |
| | Momentum | - |
| | Weight decay factor | - |
| | Optimizer | Adam |
| CIFAR-10 | Network | RseNet20 VGG16 |
| | Batch Size | 128 |
| | Epoch | 300 |
| | Learning Rate (LR) | $1 \times 10^{-1}$ |
| | Momentum | 0.9 |
| | Weight decay factor | 0.0005 |
| | Optimizer | SGD |

*b) Target detection on AeroRIT dataset:* The experiments are conducted using the large-scale dataset with the same environmental settings as in Section V-A. The experimental parameters and network architecture in this experiment are shown in Table II, and the quantitative comparisons are displayed in Table III. The GCC again demonstrates its ability to obtain a high AUC of 95.98%, with up to 90% gradient reduction, which indicates that compressing gradient with GCC is relatively compact and efficient. Furthermore, GCC achieves 0.00% AUC drop and 0.00% FPR rise, which realizes lossless gradient compression with a 20% compression rate.

The false-color image, ground truth of the target and background, and the detection results are shown in Fig. 12. It is easy to observe that the detection results obtained by our method are close to the ground truth at such a high compression rate, which indicates the considerable stability of the proposed method.

*2) Image Classification on Natural Image Dataset:*

*a) ResNet-20 on CIFAR-10 dataset:* We have motivated GCC as being more robust to different tasks because it focuses on the most replaceable elements, i.e., those that

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

TABLE III
QUANTITATIVE COMPARISONS OF DIFFERENT GRADIENT COMPRESSION METHODS ON TWO HSI DATASETS. ΔAUC(%) ↓ DENOTES THE INCREMENT OF AUC SCORE DROP

| Method | Compression Rate(CR) | Dataset | AUC(%) | Δ AUC(%)↓ | FPR(%) |
|---|---|---|---|---|---|
| Baseline | - | Xiongan New Area (Matiwan Village) | 94.10 | - | 7.93 |
| | | AeroRIT | 95.99 | - | 1.16 |
| Top-K [61] | 0.5 | Xiongan New Area (Matiwan Village) | 93.38 | 0.72 | **7.36** |
| DGC [46] | | | 92.63 | 1.47 | 7.76 |
| PowerSGD [22] | | | 93.96 | 0.14 | 8.15 |
| FedKD [62] | | | 94.03 | 0.07 | 12.88 |
| Gradient Sampling [63] | | | 92.66 | 1.44 | 7.87 |
| IntSGD [64] | | | 93.87 | 0.23 | 9.14 |
| QL-SGD [19] | | | 93.68 | 0.42 | 8.79 |
| AC-SGD [65] | | | 90.26 | 3.84 | 10.81 |
| JointSQ [66] | | | 94.08 | 0.02 | 7.64 |
| MSGC [50] | | | 93.78 | 0.32 | 8.52 |
| Top-K [61] | 0.5 | AeroRIT | 95.26 | 0.73 | 1.26 |
| DGC [46] | | | 94.12 | 1.87 | 1.23 |
| PowerSGD [22] | | | 95.05 | 0.94 | 1.16 |
| FedKD [62] | | | 95.96 | 0.03 | 3.72 |
| Gradient Sampling [63] | | | 94.24 | 1.75 | 1.17 |
| IntSGD [64] | | | 95.87 | 0.12 | 1.38 |
| QL-SGD [19] | | | 95.14 | 0.85 | 1.38 |
| AC-SGD [65] | | | 91.98 | 4.01 | 2.92 |
| JointSQ [66] | | | 95.98 | 0.01 | 1.25 |
| [50] | | | 95.12 | 0.87 | 2.01 |
| Ours(GCC) | 0.2 | Xiongan New Area (Matiwan Village) | **94.10** | **0.00** | 7.94 |
| | | AeroRIT | **95.99** | **0.00** | 1.16 |
| | 0.5 | Xiongan New Area (Matiwan Village) | 94.09 | 0.01 | 7.67 |
| | | AeroRIT | 95.98 | 0.01 | 1.15 |
| | 0.8 | Xiongan New Area (Matiwan Village) | 92.35 | 1.75 | 7.50 |
| | | AeroRIT | 95.98 | 0.01 | 1.15 |
| | 0.9 | Xiongan New Area (Matiwan Village) | 90.41 | 3.69 | 7.11 |
| | | AeroRIT | 95.98 | 0.01 | **1.14** |

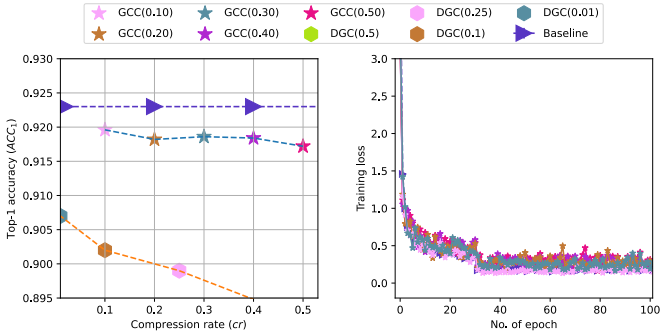
TABLE IV
QUANTITATIVE RESULTS OF RESNET20 ON CIFAR-10

| Method | Gradient Sampling | QL-SGD | CSER | Ours |
|---|---|---|---|---|
| $\mathbf{ACC_1}$ | 91.80 | 91.50 | 91.48 | **92.20** |

are absolutely unimportant, and so GCC can be extended to new tasks. To verify this, we compare GCC with DGC [46] using the CIFAR-10 dataset for extended natural image classification. The details of parameter settings and network architecture are shown in Table II. For our comparison, we use ResNet20 [67] for the image classification task displayed in Fig. 13. It can be observed from Fig. 13(a) that GCC outperforms the others and the difference from baseline is almost unchanged, where the compression rate ranges from 10% to 50%. In addition, the training loss curves on various compression rates are almost smooth, which indicates that the training process with the proposed GCC method is stable and ResNet20 converges quickly (∼50 epochs), as illustrated in Fig. 13(b).

Then, Table IV lists another comparison with other mainstream methods including Gradient Sampling [63], QL-SGD[1] [19], and CSER[2] [68], where our method outperforms others. These phenomena again demonstrate the superiority of removing gradients closest to the centroid and the validity of applying GCC in a distributed system with residual blocks.

[1]https://github.com/karakusc/horovod/tree/qsparselocal
[2]https://github.com/xcgoner/NeurIPS2020_CSER




(a) Top-1 accuracy *vs.* Compression rate (b) Training loss *vs.* No. of epoch

Fig. 13. (a) Comparison of Top-1 test accuracy with different gradient compression methods and (b) training loss of ResNet20 under different compression rates on CIFAR-10 dataset.

TABLE V
QUANTITATIVE RESULTS OF VGG16 ON CIFAR-10

| CR | - | 0.1 | 0.2 | 0.3 | 0.4 | 0.5 |
|---|---|---|---|---|---|---|
| Top-1 % | 95.62 | 95.68 | 95.62 | 95.08 | 95.30 | 94.88 |

*b) VGG16 on CIFAR-10 dataset:* A variation of VGG16 [69] is used in this experiment, and Table V shows the performance of different compression rates. It is observed that the general model again achieves lossless Top-1 accuracy with a gradient compression rate (20%). Meanwhile, the better-than-baseline performance can be attributed to the inherent nature of CIFAR-10 dataset, resulting in poorer gradient update direction, which is thankfully corrected by our proposed GCC.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

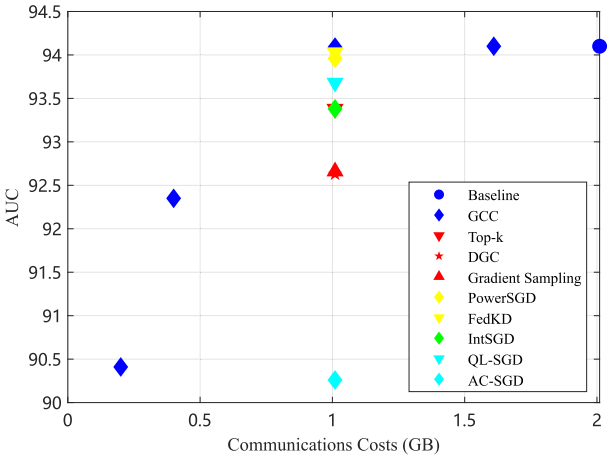



Fig. 14. Comparison of different methods in communication costs and AUC.

It is worth noting that a similar phenomenon is reported in [46]. Hence, GCC can also be applied in a plain distributed system.

*3) Communications Costs:* Accelerating large-scale model training is the basic purpose of gradient compression methods, so we analyze the actual acceleration performance of different methods. Since source code implementations of different methods may lead differences in training time, theoretical communication costs and test accuracy curves are shown in Fig. 14. It is obvious that our method can reduce the communication cost from 2.01 to 1.01 GB while maintaining the detection performance. The other comparison methods show varying degrees of degradation. In particular, compared to adaptively compressed stochastic gradient descent (AC-SGD), our method reduces the communication cost by 0.80 GB while improving the detection performance by 0.15%.

### D. Ablation Study

In this section, ablation experiments are performed to explicitly analyze the importance of each component in our proposed method. We exploit the distributed AAE learning from the large-scale Xiongan New Area (Matiwan Village) dataset for the HTD task, whose environments of hardware and software are the same as those in Section V-A. Here, two edge nodes are sufficient for verification of the gradient accumulation when training and optimizing the AAE network.

*1) Gradient Accumulation:* Table VI reports the results of several design components. Using gradient accumulation on two edge nodes, the AAE network archives the best AUC score (94.10%) and consumes the least training time (40.79 s) in one epoch, which outperforms its variants and indicates the effectiveness (better compression) and efficiency (valid acceleration) of the components.

*2) Variants of GCC:* Two variants are proposed to demonstrate the appropriateness of pruning the gradients that are close to the centroid, including: 1) *Random*: gradients are randomly pruned and 2) *Reverse*: gradients far-away centroid are pruned. The compression rate is set to be the same as that with 94.09% AUC score of GCC in Table III. We report the corresponding AUC score and FPR for each variant in Table VII. Notably, GCC outperforms its variants and *Reverse* performs the worst, demonstrating that gradients close to the centroid contain less information and can be safely removed.

TABLE VI

"✗" AND "✓" DENOTE THE EXPERIMENTS WITHOUT AND WITH THE COMPONENT, GRADIENT ACCUMULATION. "TRAINING TIME" DENOTES THE COMPUTING TIME FOR TRAINING THE AAE NETWORK IN ONE EPOCH

| Gradient Accumulation | No. of Edge Nodes | Training Time(s) | AUC (%) | FPR (%) |
|---|---|---|---|---|
| ✗ | 1 | 53.91 | 91.57 | 6.67 |
| ✗ | 2 | 47.82 | 91.43 | 8.05 |
| ✓ | 1 | 53.56 | 94.09 | 8.48 |
| ✓ | 2 | **40.79** | **94.10** | 7.11 |

TABLE VII

AUC AND FPR FOR VARIANTS OF GCC

| Variants | AUC (%) | FPR (%) |
|---|---|---|
| Baseline | **94.10** | 7.93 |
| Random | 83.58 | 8.90 |
| Reverse | 82.86 | 7.74 |
| GCC | 94.09 | **7.67** |

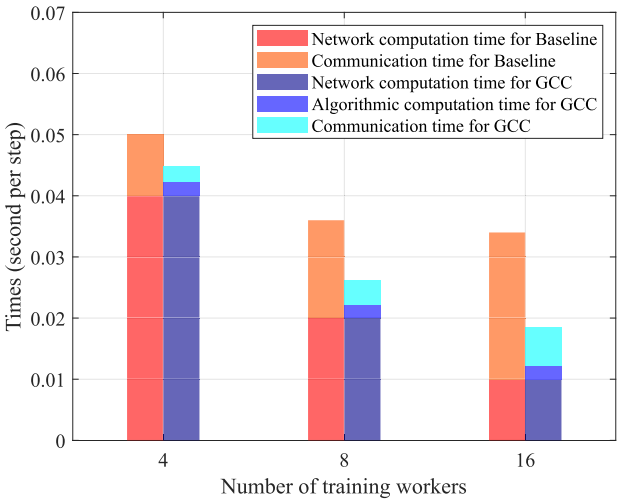



Fig. 15. Distribution of computation and communication times required per iteration when training on the Xiongan dataset.

*3) Verification on the Multinode Distributed System:* As shown in Fig. 15, in order to validate the performance of GCC on a distributed system with more workers, we train on the Xiongan dataset and record the computational and communication costs during the training process. The experimental results show that the communication time increases with the number of workers. In particular, when the gradient compression method is not applied, the training time at 16 workers is similar to that at eight workers, and a negligible boost is obtained. With the use of GCC, the communication time is significantly reduced and the time gain obtained is almost linearly related to the number of workers. In addition, the compression process of GCC consumes very little time compared to the reduced communication time, which supports its large-scale application.

## VI. CONCLUSION

In this article, we propose a novel distributed background learning with gradient compression for HTD. The main

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

attention of our method is to provide great feasibility for the interpretation of high-volume remote sensing images over 3 GB on IoT/edge devices such as satellites where computing and communication resources are constrained. To this end, a distributed background learning model with gradient accumulation is designed for HTD to avoid boundary effects and frequent communication among nodes. Then, we propose a novel gradient compression strategy via the centroid of the gradient tensor space, i.e., GCC, to reduce the communication load. Different from previous methods, it explicitly takes into consideration redundant relationships between gradients. By doing so, it can maintain target detection accuracy consistent with the benchmark after reducing the gradient by 90% on the ReroRIT dataset, thus saving communication bandwidth. The GCC can also be generalized to image classification task, achieving the state-of-the-art performance. The evaluation results show that our proposed GCC achieves higher accuracy for image classification with ResNet20 at a compression rate of 0.5. Furthermore, the designed Docker container of our method offers great feasibility for practical application.

## REFERENCES


[1] A. Toker et al., "DynamicEarthNet: Daily multi-spectral satellite dataset for semantic change segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 21126–21135.

[2] Y. Cai et al., "Mask-guided spectral-wise transformer for efficient hyperspectral image reconstruction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 17481–17490.

[3] A. Zare, C. Jiao, and T. Glenn, "Discriminative multiple instance hyperspectral target characterization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 10, pp. 2342–2354, Oct. 2018.

[4] K. Jiang, W. Xie, J. Lei, T. Jiang, and Y. Li, "LREN: Low-rank embedded network for sample-free hyperspectral anomaly detection," in *Proc. Assoc. Advancement Artif. Intell. (AAAI)*, vol. 35, May 2021, pp. 4139–4146.

[5] W. Xie, X. Zhang, Y. Li, J. Lei, J. Li, and Q. Du, "Weakly supervised low-rank representation for hyperspectral anomaly detection," *IEEE Trans. Cybern.*, vol. 51, no. 8, pp. 3889–3900, Aug. 2021.

[6] W. Xie, X. Zhang, Y. Li, K. Wang, and Q. Du, "Background learning based on target suppression constraint for hyperspectral target detection," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 5887–5897, 2020.

[7] Y. Lu, K. Xie, G. Xu, H. Dong, C. Li, and T. Li, "MTFC: A multi-GPU training framework for cube-CNN-based hyperspectral image classification," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 4, pp. 1738–1752, Oct. 2021.

[8] J. Li, Z. Liu, X. Lei, and L. Wang, "Distributed fusion of heterogeneous remote sensing and social media data: A review and new developments," *Proc. IEEE*, vol. 109, no. 8, pp. 1350–1363, Aug. 2021.

[9] D. Miner and A. Shook, *MapReduce Design Patterns: Building Effective Algorithms and Analytics for Hadoop and Other Systems*. Sebastopol, CA, USA: O'Reilly Media, 2012.

[10] Z. Wu, J. Sun, Y. Zhang, Z. Wei, and J. Chanussot, "Recent developments in parallel and distributed computing for remotely sensed big data processing," *Proc. IEEE*, vol. 109, no. 8, pp. 1282–1305, Aug. 2021.

[11] J. M. Haut, M. E. Paoletti, S. Moreno-Alvarez, J. Plaza, J.-A. Rico-Gallego, and A. Plaza, "Distributed deep learning for remote sensing data interpretation," *Proc. IEEE*, vol. 109, no. 8, pp. 1320–1349, Aug. 2021.

[12] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[13] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2012, pp. 1097–1105.

[14] D. Alistarh, D. Grubic, J. Z. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 1707–1718.

[15] S. P. Karimireddy, Q. Rebjock, S. Stich, and M. Jaggi, "Error feedback fixes SignSGD and other gradient compression schemes," in *Proc. Int. Conf. Mach. Learn.*, vol. 97, Jun. 2019, pp. 3252–3261.

[16] C. Chen et al., "ScaleCom: Scalable sparsified gradient compression for communication-efficient distributed training," in *Proc. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, Jan. 2020, pp. 13551–13563.

[17] J. Wangni, J. Wang, J. Liu, and T. Zhang, "Gradient sparsification for communication-efficient distributed optimization," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, 2018, pp. 1299–1309.

[18] N. Ivkin, D. Rothchild, E. Ullah, V. Braverman, I. Stoica, and R. Arora, "Communication-efficient distributed SGD with sketching," in *Proc. Neural Inf. Process. Syst. (NeurIPS)*, Jan. 2019, pp. 13142–13152.

[19] D. Basu, D. Data, C. Karakus, and S. N. Diggavi, "Qsparse-local-SGD: Distributed SGD with quantization, sparsification and local computations," in *Proc. Adv. Neural Inf. Process. Syst., Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 14668–14679.

[20] H. Lim, D. G. Andersen, and M. Kaminsky, "3LC: Lightweight and effective traffic compression for distributed machine learning," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 1, Apr. 2019, pp. 53–64.

[21] M. Cho, V. Muthusamy, B. Nemanich, and R. Puri, "GradZip: Gradient compression using alternating matrix factorization for large-scale deep learning," in *Proc. Neural Inf. Process. Syst. (NeurIPS)*, 2019.

[22] T. Vogels, S. P. Karimireddy, and M. Jaggi, "PowerSGD: Practical low-rank gradient compression for distributed optimization," in *Proc. Neural Inf. Process. Syst. (NeurIPS)*, Jan. 2019, pp. 14236–14245.

[23] Y. Li et al., "A network-centric hardware/algorithm co-design to accelerate distributed training of deep neural networks," in *Proc. 51st Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2018, pp. 175–188.

[24] J. Dean, "Large scale distributed deep networks," in *Proc. Neural Inf. Process. Syst. (NeurIPS)*, vol. 2, 2012, pp. 1223–1231.

[25] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *J. Parallel Distrib. Comput.*, vol. 69, no. 2, pp. 117–124, Feb. 2009.

[26] Y. Jiang, H. Gu, Y. Lu, and X. Yu, "2D-HRA: Two-dimensional hierarchical ring-based all-reduce algorithm in large-scale distributed machine learning," *IEEE Access*, vol. 8, pp. 183488–183494, 2020.

[27] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "ASTRA-SIM: Enabling SW/HW co-design exploration for distributed DL training platforms," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, Aug. 2020, pp. 81–92.

[28] S. Wang, D. Li, J. Geng, Y. Gu, and Y. Cheng, "Impact of network topology on the performance of DML: Theoretical analysis and practical factors," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1729–1737.

[29] M. Wang et al., "Neural network meets DCN: Traffic-driven topology adaptation with deep learning," *ACM Meas. Anal. Comput. Syst.*, vol. 2, no. 2, pp. 1–25, 2018.

[30] S. Alqahtani and M. Demirbas, "Performance analysis and comparison of distributed machine learning systems," 2019, *arXiv:1909.02061*.

[31] M. N. H. Nguyen et al., "Self-organizing democratized learning: Toward large-scale distributed learning systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 10698–10710, Dec. 2023.

[32] V. A. A. Quirita et al., "A new cloud computing architecture for the classification of remote sensing data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 10, no. 2, pp. 409–416, Feb. 2017.

[33] A. Sergeev and M. D. Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," 2018, *arXiv:1802.05799*.

[34] Z. Wu, Y. Li, A. Plaza, J. Li, F. Xiao, and Z. Wei, "Parallel and distributed dimensionality reduction of hyperspectral data on cloud computing architectures," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 6, pp. 2270–2278, Jun. 2016.

[35] Z. Wu et al., "Scheduling-guided automatic processing of massive hyperspectral image classification on cloud computing architectures," *IEEE Trans. Cybern.*, vol. 51, no. 7, pp. 3588–3601, Jul. 2021.

[36] Q. Du, B. Tang, W. Xie, and W. Li, "Parallel and distributed computing for anomaly detection from hyperspectral remote sensing imagery," *Proc. IEEE*, vol. 109, no. 8, pp. 1306–1319, Aug. 2021.

[37] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Deep learning classifiers for hyperspectral imaging: A review," *ISPRS J. Photogramm. Remote Sens.*, vol. 158, pp. 279–317, Dec. 2019.

[38] J. M. Haut et al., "Cloud deep networks for hyperspectral image analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 12, pp. 9832–9848, Dec. 2019.


Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.

[39] H. Xu et al., "GRACE: A compressed communication framework for distributed machine learning," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jul. 2021, pp. 561–572.

[40] N. Strom, "Scalable distributed DNN training using commodity GPU cloud computing," in *Proc. Interspeech*, Sep. 2015, pp. 1488–1492.

[41] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-IID data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, Sep. 2019.

[42] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, "Sparsified SGD with memory," in *Proc. Neural Inf. Process. Syst. (NeurIPS)*, vol. 31, Dec. 2018, pp. 4447–4458.

[43] A. M. Abdelmoniem, A. Elzanaty, M. Alouini, and M. Canini, "An efficient statistical-based gradient compression technique for distributed training systems," in *Proc. Mach. Learn. Syst. (MLSys)*, Jan. 2021, pp. 297–322.

[44] Y. Tsuzuku, H. Imachi, and T. Akiba, "Variance-based gradient compression for efficient distributed deep learning," 2018, *arXiv:1802.06058*.

[45] S. Wiedemann, K.-R. Müller, and W. Samek, "Compact and computationally efficient representation of deep neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 3, pp. 772–785, Mar. 2020.

[46] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Feb. 2018.

[47] L. Abrahamyan, Y. Chen, G. Bekoulis, and N. Deligiannis, "Learned gradient compression for distributed deep learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 12, pp. 7330–7344, Dec. 2022.

[48] X. Zhou, L. Chang, and J. Cao, "Communication-efficient nonconvex federated learning with error feedback for uplink and downlink," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Nov. 23, 2024, doi: 10.1109/TNNLS.2023.3333804.

[49] X. Zhou and X. Wang, "Memory and communication efficient federated kernel $k$-means," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 5, pp. 7114–7125, May 2022.

[50] R. Jin, Y. Liu, Y. Huang, X. He, T. Wu, and H. Dai, "Sign-based gradient descent with heterogeneous data: Convergence and byzantine resilience," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jan. 12, 2024, doi: 10.1109/TNNLS.2023.3345367.

[51] S. Li et al., "PyTorch distributed: Experiences on accelerating data parallel training," 2020, *arXiv:2006.15704*.

[52] G. Yu et al., "IronForge: An open, secure, fair, decentralized federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Nov. 21, 2024, doi: 10.1109/TNNLS.2023.3329249.

[53] Y. Peng et al., "A generic communication scheduler for distributed DNN training acceleration," in *Proc. 27th ACM Symp. Operating Syst. Princ.*, Oct. 2019, pp. 16–29.

[54] P. Thomas Fletcher, S. Venkatasubramanian, and S. Joshi, "Robust statistics on Riemannian manifolds via the geometric median," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2008, pp. 1–8.

[55] A. Paszke et al., "Automatic differentiation in PyTorch," 2017.

[56] J. A. Swets, R. M. Dawes, and J. Monahan, "Better decisions through science," *Sci. Amer.*, vol. 283, no. 4, pp. 82–87, Oct. 2000.

[57] A. P. Bradley, "The use of the area under the ROC curve in the evaluation of machine learning algorithms," *Pattern Recognit.*, vol. 30, no. 7, pp. 1145–1159, Jul. 1997.

[58] Y. Cen et al., "Aerial hyperspectral remote sensing classification dataset of Xiongan New Area (Matiwan Village)," *J. Remote Sens.*, vol. 24, no. 11, pp. 1299–1306, 2020.

[59] A. Rangnekar, N. Mokashi, E. J. Ientilucci, C. Kanan, and M. J. Hoffman, "AeroRIT: A new scene for hyperspectral image analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 11, pp. 8116–8124, Nov. 2020.

[60] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep. TR-2009, 2009.

[61] A. Fikri Aji and K. Heafield, "Sparse communication for distributed gradient descent," 2017, *arXiv:1704.05021*.

[62] C. Wu, F. Wu, L. Lyu, Y. Huang, and X. Xie, "Communication-efficient federated learning via knowledge distillation," *Nature Commun.*, vol. 13, no. 1, p. 2032, Apr. 2022.

[63] L. Song et al., "Communication efficient SGD via gradient sampling with Bayes prior," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 12060–12069.

[64] K. Mishchenko, B. Wang, D. Kovalev, and P. Richtárik, "IntSGD: Adaptive floatless compression of stochastic gradients," 2021, *arXiv:2102.08374*.

[65] G. Yan, T. Li, S.-L. Huang, T. Lan, and L. Song, "AC-SGD: Adaptively compressed SGD for communication-efficient distributed learning," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 9, pp. 2678–2693, Sep. 2022.

[66] W. Xie et al., "JointSQ: Joint sparsification-quantization for distributed learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 5778–5787.

[67] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[68] C. Xie, S. Zheng, O. Koyejo, I. Gupta, M. Li, and H. Lin, "CSER: Communication-efficient SGD with error reset," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2020, pp. 12593–12603.

[69] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. Peter Graf, "Pruning filters for efficient ConvNets," 2016, *arXiv:1608.08710*.



**Weiying Xie** (Senior Member, IEEE) received the Ph.D. degree in communication and information systems from Xidian University, Xi'an, China, in 2017.

She is currently a Professor with the State Key Laboratory of Integrated Services Networks, Xidian University. She has authored over 50 articles in refereed journals and proceedings, including IEEE TRANSACTIONS ON IMAGE PROCESSING, IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON CYBERNETICS, Conference on IEEE/CVF Computer Vision and Pattern Recognition (CVPR), and Association for the Advancement of Artificial Intelligence (AAAI). Her research interests include neural networks, machine learning, hyperspectral image processing, and high-performance computing.



**Jitao Ma** received the B.E. degree in telecommunications engineering from Xidian University, Xi'an, China, in 2021, where he is currently pursuing the Ph.D. degree with the Image Coding and Processing Center, State Key Laboratory of Integrated Services Networks.

His research interests include remote sensing image processing, distributed deep learning, and anomaly detection.



**Tianen Lu** received the B.E. degree in electronics and information engineering from Changchun University of Science and Technology, Changchun, China, in 2021, and the M.S. degree in telecommunication and information systems from Xidian University, Xi'an, China, in 2024.

His research interests include deep learning and distributed computing.



**Yunsong Li** (Member, IEEE) received the M.S. degree in telecommunication and information systems and the Ph.D. degree in signal and information processing from Xidian University, Xi'an, China, in 1999 and 2002, respectively.

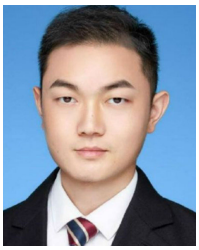
In 1999, he joined the School of Telecommunications Engineering, Xidian University, where he is currently a Professor and also the Director of the Image Coding and Processing Center, State Key Laboratory of Integrated Services Networks. His research interests include image and video processing and high-performance computing.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.



**Jie Lei** (Member, IEEE) received the M.S. degree in telecommunication and information systems and the Ph.D. degree in signal and information processing from Xidian University, Xi'an, China, in 2006 and 2010, respectively.

He was a Visiting Scholar with the Department of Computer Science, University of California at Los Angeles, Los Angeles, CA, USA, from 2014 to 2015. He is currently an Associate Professor with the School of Telecommunications Engineering, Xidian University, where he is also a member of the Image Coding and Processing Center, State Key Laboratory of Integrated Services Networks. He is also with the Science and Technology on Electro-Optic Control Laboratory, Luoyang, China. His research interests focus on image and video processing, computer vision, and customized computing for big-data applications.



**Leyuan Fang** (Senior Member, IEEE) received the Ph.D. degrees from the College of Electrical and Information Engineering, Hunan University, Changsha, China, in 2008 and 2015, respectively.

From 2011 to 2012, with the support of China Scholarship Council, he was a Visiting Ph.D. Student with the Department of Ophthalmology, Duke University, Durham, NC, USA. From 2016 to 2017, he was a Post-Doctoral Research Fellow with the Department of Biomedical Engineering, Duke University. Since 2017, he has been an Associate Professor with the College of Electrical and Information Engineering, Hunan University. His research interests include sparse representation and multiresolution analysis in remote sensing and medical image processing.

Dr. Fang won the Scholarship Award for Excellent Doctoral Student granted by the Chinese Ministry of Education in 2011.



**Qian Du** (Fellow, IEEE) received the Ph.D. degree in electrical engineering from the University of Maryland, Baltimore, MD, USA, in 2000.

She is currently the Bobby Shackouls Professor with the Department of Electrical and Computer Engineering, Mississippi State University, Starkville, MS, USA. She is also an Adjunct Professor with the College of Surveying and Geo-Informatics, Tongji University, Shanghai, China. Her research interests include hyperspectral remote sensing image analysis and applications, pattern classification, data compression, and neural networks.

Dr. Du is a fellow of the SPIE-International Society for Optics and Photonics. She was a recipient of the 2010 Best Reviewer Award from the IEEE Geoscience and Remote Sensing Society. She was a Co-Chair of the Data Fusion Technical Committee of the IEEE Geoscience and Remote Sensing Society from 2009 to 2013, and the Chair of the Remote Sensing and Mapping Technical Committee of the International Association for Pattern Recognition from 2010 to 2014. She has served as an Associate Editor for IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING, *Journal of Applied Remote Sensing*, and IEEE SIGNAL PROCESSING LETTERS. Since 2016, she has been the Editor-in-Chief of IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:45:01 UTC from IEEE Xplore. Restrictions apply.